

Hands-on Activity 7.1 Functions in JavaScript	
Course Code: CPE025 B	Program: BSCPE
Course Title: Software Design 2	Date Performed: 03/02/2026
Section: CPE32S4	Date Submitted: 03/02/2026
Name: Bautista, Jhon Hendricks	Instructor: Engr. Neal Barton James Matira
1. Discussion	
Functions act as fundamental elements of programming across most languages. Writing complex programs without them remains impractical. Developers frequently utilize functions created by others, ranging from built-in language features like <code>console.log</code> to third-party solutions. Libraries such as Leaflet, D3.js, and jQuery bundle these functions to solve specific classes of problems. Beyond using pre-existing tools, developers write custom functions to logically divide programs, facilitate code reuse, and isolate parts for testing. JavaScript offers various methods to define functions, including classical declarations, function expressions, and arrow functions. These structures handle parameter passing and value returns. Key concepts include the distinction between named and anonymous functions, the utility of callback functions, and the implementation of recursion. A strong grasp of these mechanics allows for the effective customization and definition of functions in software development.	
2. Materials and Equipment	
Computer Integrated Development Environment Internet	
3. Procedure	

5.1.11 LAB Functions (Part 1)

Estimated time

30-45 minutes

Level of difficulty

Medium

Objectives

Familiarize the student with:

- Function basics (what are functions, declaring functions, calling functions, local variables, the return statement, function parameters, shadowing);
- Functions as first-class members (function expressions, passing a function as a parameter, callbacks);
- Arrow functions (declaring and calling);
- Recursion (basic idea).

Scenario

Our program has grown quite a bit, making it a little hard to read. It is especially visible in the switch instruction, where most of the logic is enclosed. Try to organize your program code by using functions. Define and call three functions in the appropriate places:

- `showContact`: the function should take two arguments; the first is the list of contacts, and the second is the index number of the contact to display; inside the function, check if the correct arguments are passed, that is, if the contacts are an array (use the `instanceof Array` construction for this);
- `showAllContacts`: the function should take one argument, the list of contacts; inside the function, check if the given argument is an array;
- `addNewContact`: the function should take four arguments, a contact list and the data of the new contact, that is: name, phone, and number; before adding a new contact, check if the passed argument is an array and if the new contact data have any value.

5.1.12 LAB Functions (Part 2)

Estimated time

30-60 minutes

Level of difficulty

Medium

Objectives

Familiarize the student with:

- Function basics (what are functions, declaring functions, calling functions, local variables, the return statement, function parameters, shadowing);
- Functions as first-class members (function expressions, passing a function as a parameter, callbacks);
- Arrow functions (declaring and calling);
- Recursion (basic idea).

Scenario

We will use the functions to add one more item of functionality. Arrays have a `sort` method that allows us to sort their elements. To this method, we pass a function which should compare two elements of the array and decide which one is smaller and which one is bigger. If the first element is smaller, the function returns a value less than zero, if they are equal it returns zero, and if the first is larger, it returns a value greater than zero. For example, the array:

4. Output

- 5.1.11 LAB

JavaScript

```
let contacts = [
  {
    name: "Maxwell Wright",
    phone: "(0191) 719 6495",
    email: "Curabitur.egestas.nunc@nonummyac.co.uk"
  },
  {
    name: "Raja Villarreal",
    phone: "0866 398 2895",
    email: "posuere.vulputate@sed.com"
  },
  {
    name: "Helen Richards",
    phone: "0800 1111",
    email: "libero@convallis.edu"
  }
];

// show a single contact by index
function showContact(contactList, index) {
  if (!Array.isArray(contactList)) {
    console.log("Error: contacts should be an array.");
    return;
  }
  if (typeof index !== "number" || index < 0 || index >=
contactList.length) {
    console.log("Error: invalid index.");
    return;
  }
  const contact = contactList[index];
  console.log(`Name: ${contact.name}`);
  console.log(`Phone: ${contact.phone}`);
  console.log(`Email: ${contact.email}`);
  console.log("-----");
}

// show all contacts
function showAllContacts(contactList) {
  if (!Array.isArray(contactList)) {
    console.log("Error: contacts should be an array.");
    return;
  }
  contactList.forEach((contact, i) => {
    console.log(`Contact #${i + 1}`);
    console.log(`Name: ${contact.name}`);
    console.log(`Phone: ${contact.phone}`);
    console.log(`Email: ${contact.email}`);
    console.log("-----");
  });
}
```

```

    });
}

// add a new contact
function addNewContact(contactList, name, phone, email) {
    if (!Array.isArray(contactList)) {
        console.log("Error: contacts should be an array.");
        return;
    }
    if (!name || !phone || !email) {
        console.log("Error: all contact data must have a value.");
        return;
    }
    contactList.push({ name, phone, email });
    console.log(`Contact ${name} added successfully!`);
}

let action = prompt(
    "Choose an action:\n" +
    "1 - Show all contacts\n" +
    "2 - Show a contact\n" +
    "3 - Add a new contact\n" +
    "4 - Exit"
);

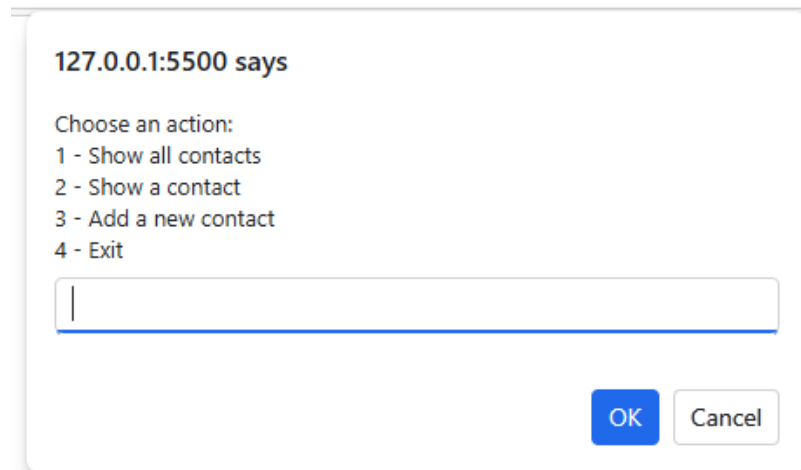
if (action === "1") {
    showAllContacts(contacts);
} else if (action === "2") {
    let index = parseInt(prompt("Enter contact index (starting from 1):"));
    showContact(contacts, index - 1);
} else if (action === "3") {
    let name = prompt("Enter name:");
    let phone = prompt("Enter phone:");
    let email = prompt("Enter email:");
    addNewContact(contacts, name, phone, email);
} else if (action === "4") {
    console.log("Program exited.");
} else {
    console.log("Invalid option selected.");
}

```

Figure 1. Javascript Code

In the first procedure problem, I followed the same logic for each option just like before but now they are in a function code structure. I am now passing arguments and having

the returning value for each option as opposed to creating the whole logic on the else if statement I am instead calling the function and performing the operation as needed.



127.0.0.1:5500 says

Choose an action:

- 1 - Show all contacts
- 2 - Show a contact
- 3 - Add a new contact
- 4 - Exit

OK Cancel

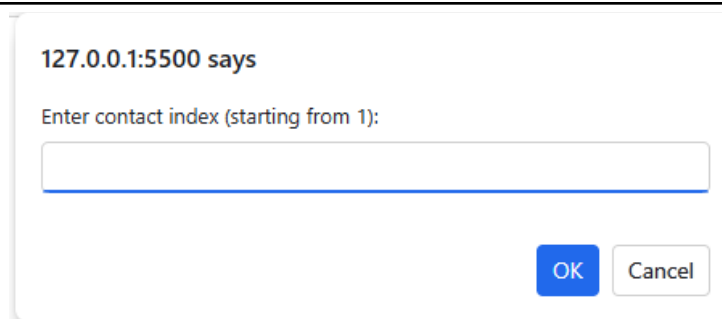
Figure 2. Main Menu

Once the program runs there is a prompt notification which is shown in figure 2 which serves as the main menu of the program.

Contact #1	5.1.11 LAB.js:43
Name: Maxwell Wright	5.1.11 LAB.js:44
Phone: (0191) 719 6495	5.1.11 LAB.js:45
Email: Curabitur.egestas.nunc@nonummyac.co.uk	5.1.11 LAB.js:46
-----	5.1.11 LAB.js:47
Contact #2	5.1.11 LAB.js:43
Name: Raja Villarreal	5.1.11 LAB.js:44
Phone: 0866 398 2895	5.1.11 LAB.js:45
Email: posuere.vulputate@sed.com	5.1.11 LAB.js:46
-----	5.1.11 LAB.js:47
Contact #3	5.1.11 LAB.js:43
Name: Helen Richards	5.1.11 LAB.js:44
Phone: 0800 1111	5.1.11 LAB.js:45
Email: libero@conwallis.edu	5.1.11 LAB.js:46
-----	5.1.11 LAB.js:47

Figure 3. Showing All Contacts

In figure 3 we can see the output of the program which is to print all the contact information per person on the array.



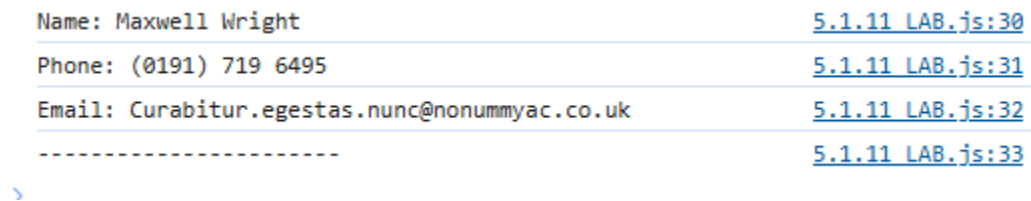
127.0.0.1:5500 says

Enter contact index (starting from 1):

OK Cancel

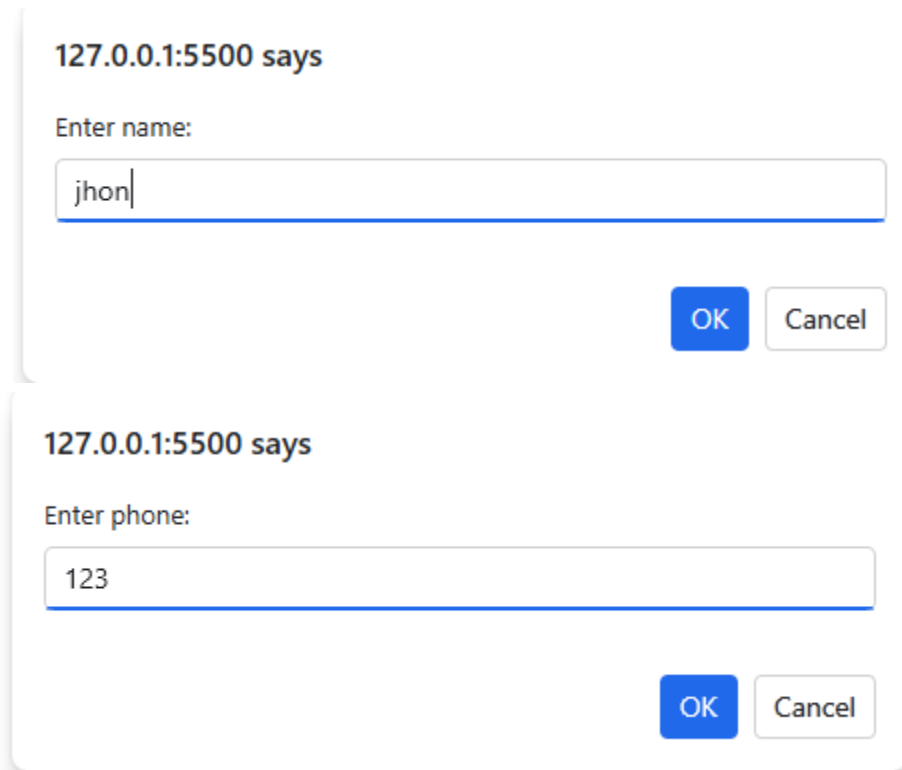
Figure 4. Showing a Contact

In figure 4 it shows the prompt when the user chooses to show a contact. It will ask for a number which will serve as the index to call the correct contact information on the array.



```
Name: Maxwell Wright 5.1.11 LAB.js:30
Phone: (0191) 719 6495 5.1.11 LAB.js:31
Email: Curabitur.egestas.nunc@nonummyac.co.uk 5.1.11 LAB.js:32
----- 5.1.11 LAB.js:33
>
```

Figure 5. Output of Showing a Contact



127.0.0.1:5500 says

Enter name:

OK Cancel

127.0.0.1:5500 says

Enter phone:

OK Cancel

127.0.0.1:5500 says

Enter email:

qjhtbautista@tip.edu.ph

OK

Cancel

Figure 6. Selecting Adding a Contact

In figure 6 we can see the prompts that will be asked on the user if they choose to add a contact. The following prompt for asking the name, contact number and email. Will require the user to fill these up in order to successfully add a contact.

Contact jhon added successfully!

[5.1.11 LAB.js:62](#)

Figure 7. Output of Adding a Contact

Figure 7 shows output of a successful adding of a contact.

Error: all contact data must have a value.

[5.1.11 LAB.js:58](#)

Figure 8. Unsuccessful Adding of Contact

Figure 8 shows the error code once a prompt is not filled and the user tries to add the contact.

Program exited.

[5.1.11 LAB.js:87](#)

Figure 9. Exiting the Program

5.1.12 LAB

JavaScript

```
let contacts = [
  {
    name: "Maxwell Wright",
    phone: "(0191) 719 6495",
    email: "Curabitur.egestas.nunc@nonummyac.co.uk"
  },
  {
    name: "Raja Villarreal",
```

```

        phone: "0866 398 2895",
        email: "posuere.vulputate@sed.com"
    },
    {
        name: "Helen Richards",
        phone: "0800 1111",
        email: "libero@convallis.edu"
    }
];

function showContact(contactList, index) {
    if (!Array.isArray(contactList)) {
        console.log("Error: contacts should be an array.");
        return;
    }
    if (typeof index !== "number" || index < 0 || index >=
contactList.length) {
        console.log("Error: invalid index.");
        return;
    }

    const contact = contactList[index];
    console.log(`Name: ${contact.name}`);
    console.log(`Phone: ${contact.phone}`);
    console.log(`Email: ${contact.email}`);
    console.log("-----");
}

function showAllContacts(contactList) {
    if (!Array.isArray(contactList)) {
        console.log("Error: contacts should be an array.");
        return;
    }

    contactList.forEach((contact, i) => {
        console.log(`Contact #${i + 1}`);
        console.log(`Name: ${contact.name}`);
        console.log(`Phone: ${contact.phone}`);
        console.log(`Email: ${contact.email}`);
        console.log("-----");
    });
}

function addNewContact(contactList, name, phone, email) {
    if (!Array.isArray(contactList)) {
        console.log("Error: contacts should be an array.");
        return;
    }
}

```



```

    if (!name || !phone || !email) {
        console.log("Error: all contact data must have a value.");
        return;
    }

    contactList.push({ name, phone, email });
    console.log(`Contact ${name} added successfully!`);
}

function sortContacts(contactList, key) {
    if (!Array.isArray(contactList)) {
        console.log("Error: contacts should be an array.");
        return;
    }

    if (!["name", "phone", "email"].includes(key)) {
        console.log("Error: Invalid sort option.");
        return;
    }

    contactList.sort((a, b) => a[key].localeCompare(b[key]));
    console.log(`Contacts sorted by ${key}.`);
}

let option = prompt(
    "Choose an option:\n1 - Show all contacts\n2 - Show one contact\n3 - Add new contact\n4 - Sort contacts"
);

switch (option) {
    case "1":
        showAllContacts(contacts);
        break;

    case "2":
        let index = parseInt(prompt("Enter contact index (starting from 0):"));
        showContact(contacts, index);
        break;

    case "3":
        let name = prompt("Enter name:");
        let phone = prompt("Enter phone:");
        let email = prompt("Enter email:");
        addNewContact(contacts, name, phone, email);
        break;

    case "4":
        let sortOption = prompt("Sort by: name, phone, or email?");

```

```
sortContacts(contacts, sortOption);  
showAllContacts(contacts);  
break;  
  
default:  
  console.log("Invalid option selected.");  
}
```

Figure 10. Javascript Code

The new javascript code in figure 10 keeps the previous functions but now it has a new option and function which is to sort the contact array based on an item option on the array. This is done by getting a prompt from the user and ask if sorting is done through the name, phone, or email. In the program this is done by checking the input if its a valid key and then passing this key onto the sort statement which calls the item and compares the two data through the localcompare.

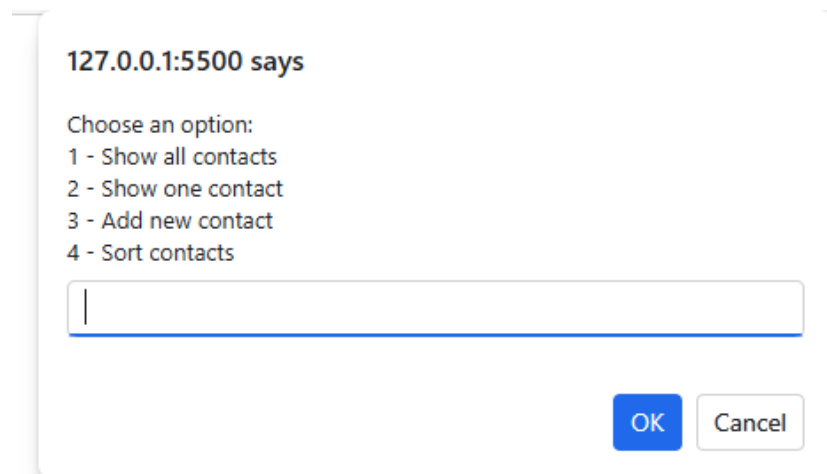
A terminal window showing a prompt from 127.0.0.1:5500. The prompt asks the user to "Choose an option:" and lists four choices: "1 - Show all contacts", "2 - Show one contact", "3 - Add new contact", and "4 - Sort contacts". Below the list is an empty text input field with a blue border. At the bottom right of the terminal window are two buttons: "OK" (blue) and "Cancel" (white with a blue border).

Figure 11. New Main Menu

Figure 11 shows the new main menu which now includes a new option to sorting of contacts.

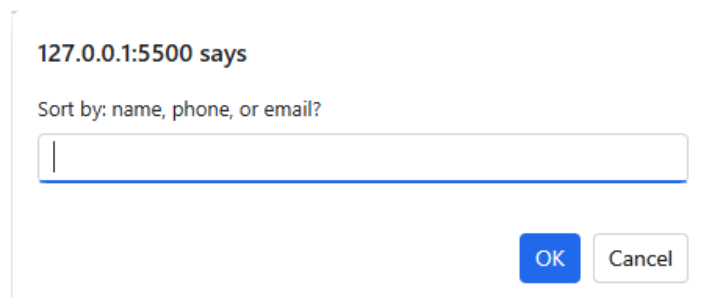
A terminal window showing a prompt from 127.0.0.1:5500. The prompt asks the user to "Sort by: name, phone, or email?". Below the text is an empty text input field with a blue border. At the bottom right of the terminal window are two buttons: "OK" (blue) and "Cancel" (white with a blue border).

Figure 12. Sorting Options

Figure 12 shows the output prompt when selecting to sort the contacts. The user can choose what item to sort the contacts by.

Contacts sorted by name.	5.1.12 LAB.js:79
Contact #1	5.1.12 LAB.js:44
Name: Helen Richards	5.1.12 LAB.js:45
Phone: 0800 1111	5.1.12 LAB.js:46
Email: libero@convallis.edu	5.1.12 LAB.js:47
-----	5.1.12 LAB.js:48
Contact #2	5.1.12 LAB.js:44
Name: Maxwell Wright	5.1.12 LAB.js:45
Phone: (0191) 719 6495	5.1.12 LAB.js:46
Email: Curabitur.egestas.nunc@nonummyac.co.uk	5.1.12 LAB.js:47
-----	5.1.12 LAB.js:48
Contact #3	5.1.12 LAB.js:44
Name: Raja Villarreal	5.1.12 LAB.js:45
Phone: 0866 398 2895	5.1.12 LAB.js:46
Email: posuere.vulputate@sed.com	5.1.12 LAB.js:47
-----	5.1.12 LAB.js:48

Figure 13. Sorting by Name

Shown in figure 13 is the sorting of the contacts through by names. It shows successful sorting of the names.

top	Filter	Default levels	NO ISSUES
Contacts sorted by phone.	5.1.12 LAB.js:79		
Contact #1	5.1.12 LAB.js:44		
Name: Maxwell Wright	5.1.12 LAB.js:45		
Phone: (0191) 719 6495	5.1.12 LAB.js:46		
Email: Curabitur.egestas.nunc@nonummyac.co.uk	5.1.12 LAB.js:47		
-----	5.1.12 LAB.js:48		
Contact #2	5.1.12 LAB.js:44		
Name: Helen Richards	5.1.12 LAB.js:45		
Phone: 0800 1111	5.1.12 LAB.js:46		
Email: libero@convallis.edu	5.1.12 LAB.js:47		
-----	5.1.12 LAB.js:48		
Contact #3	5.1.12 LAB.js:44		
Name: Raja Villarreal	5.1.12 LAB.js:45		
Phone: 0866 398 2895	5.1.12 LAB.js:46		
Email: posuere.vulputate@sed.com	5.1.12 LAB.js:47		
-----	5.1.12 LAB.js:48		

Figure 14. Sorting by Phone Number

Figure 14 shows the sorting of the contacts through the phone number. It shows the successful sorting of the phone number.

Contacts sorted by email.	5.1.12 LAB.js:79
Contact #1	5.1.12 LAB.js:44
Name: Maxwell Wright	5.1.12 LAB.js:45
Phone: (0191) 719 6495	5.1.12 LAB.js:46
Email: Curabitur.egestas.nunc@nonummyac.co.uk	5.1.12 LAB.js:47
-----	5.1.12 LAB.js:48
Contact #2	5.1.12 LAB.js:44
Name: Helen Richards	5.1.12 LAB.js:45
Phone: 0800 1111	5.1.12 LAB.js:46
Email: libero@convallis.edu	5.1.12 LAB.js:47
-----	5.1.12 LAB.js:48
Contact #3	5.1.12 LAB.js:44
Name: Raja Villarreal	5.1.12 LAB.js:45
Phone: 0866 398 2895	5.1.12 LAB.js:46
Email: posuere.vulputate@sed.com	5.1.12 LAB.js:47
-----	5.1.12 LAB.js:48

Figure 15. Sorting by Email

5. Supplementary Activity

Scenario:

You are building a Global Technical Support Team directory. This version is more advanced than previous labs because the program must stay active, allowing the user to perform multiple tasks in one session without restarting the script. Each contact now includes a "skills" array and an "availability" status.

Dataset:

Name	Role	Skills	Availability
Linus Torvalds	System Admin	Linux, Git, Kernels	TRUE
Ada Lovelace	Logic Analyst	Algorithms, Math, Analytics	FALSE
Alan Turing	Cryptographer	Logic, Enigma, Security	TRUE

Tasks

Initialize the Data: Create an array named contacts containing three objects using the data in the table above.

- The Persistent Loop:** Enclose your entire program logic in a loop. The program should continue to ask the user for an action until they type "quit".
- The Interaction Menu:** Use a prompt to ask the user which action they want to perform. The options are:
 - show:** Ask the user for an index number. If the index is valid, display the name, role, and the first skill from that contact's skills array.
 - all:** Use a loop to display the name of every contact in the list.
 - add:** Prompt the user for a Name, Role, and a single Skill. Validation Requirement: Use an if statement to ensure none of the fields are empty. Only if all data is present should you push the new object into the array.
 - search:** Prompt the user for a Name. Use a loop to find the contact. If found, display their Role and whether they are currently "Available" or "Busy" based on their boolean value.
- Completion:** If the user enters "quit", use an alert to say goodbye and end the loop using the break statement.

Include Screenshots and Analysis of your Output.

OUTPUT

JavaScript

```
let contacts = [
  {
    name: "Linus Torvalds",
    role: "System Admin",
    skills: ["Linux", "Git", "Kernels"],
    availability: true
  },
  {
    name: "Ada Lovelace",
    role: "Logic Analyst",
    skills: ["Algorithms", "Math", "Analytics"],
    availability: false
  },
  {
    name: "Alan Turing",
    role: "Cryptographer",
    skills: ["Logic", "Enigma", "Security"],
    availability: true
  }
];

function showContact(contactList) {
  let index = prompt("Enter contact index (0-" + (contactList.length - 1) + "):");

  if (index === null) return; // error check

  index = Number(index);

  if (!isNaN(index) && index >= 0 && index < contactList.length) {

    console.log("Contact Found:");
    console.log("Name:", contactList[index].name);
    console.log("Role:", contactList[index].role);
    console.log("First Skill:", contactList[index].skills[0]); // first skill index

  } else {
    console.log("Invalid index.");
  }
}

function showAllContacts(contactList) {
  console.log("All Contacts:");
  for (let i = 0; i < contactList.length; i++) {
    console.log(contactList[i].name); // print lang
  }
}
```

```

}

function addContact(contactList) {
  let name = prompt("Enter Name:");
  let role = prompt("Enter Role:");
  let skill = prompt("Enter a Skill:");

  if (!name || !role || !skill) {
    console.log("Error: All fields must be filled."); // checker of
empty fields
    return;
  }

  contactList.push({
    name: name,
    role: role,
    skills: [skill],
    availability: true
  });

  console.log("Contact added successfully.");
}

function searchContact(contactList) {
  let searchName = prompt("Enter name to search:");
  if (!searchName) return;

  let found = false;

  for (let i = 0; i < contactList.length; i++) {
    if (contactList[i].name.toLowerCase() === searchName.toLowerCase())
    {
      console.log("Contact Found:");
      console.log("Role:", contactList[i].role);
      console.log(
        "Status:",
        contactList[i].availability ? "Available" : "Busy"
      );

      found = true;
      break;
    }
  }

  if (!found) {
    console.log("Contact not found.");
  }
}

```

```

// while loops for no end
while (true) {

    let option = prompt(
        "Choose an action:\n" +
        "show - Show a contact\n" +
        "all - Show all contacts\n" +
        "add - Add new contact\n" +
        "search - Search by name\n" +
        "quit - Exit program"
    );

    if (option === null) continue;

    if (option === "show") {
        showContact(contacts);
    } else if (option === "all") {
        showAllContacts(contacts);
    } else if (option === "add") {
        addContact(contacts);
    } else if (option === "search") {
        searchContact(contacts);
    } else if (option === "quit") {
        alert("PROGRAM END");
        break;
    } else {
        console.log("Invalid option.");
    }
}

```

Figure 16. Supplementary Program Code

In figure 16 we can see the implemented program to perform the needed tasks in the supplementary activity instructions. I kept most of the logic of the functions from the previous procedure activity since it has almost the same logic and needs wherein I implement if else statements to check the option selection and then use indexing to output the needed information from the array. For the use of functions, I mainly passed the array since this is where the data handling and modification needed for each option takes place. The main difference is that now I am able to implement the use of functions in an infinite menu loop. The use of logical and comparison operators are also implemented to check the value of the items on each of the option operations.

127.0.0.1:5500 says

Choose an action:

- show - Show a contact
- all - Show all contacts
- add - Add new contact
- search - Search by name
- quit - Exit program

show

OK Cancel

Figure 17. New Main Menu

Figure 17 shows the new main menu for the program wherein the users can select from showing a contact, showing all, adding and searching a contact and ending the program.

127.0.0.1:5500 says

Enter contact index (0-2):

1

OK Cancel

Figure 18. Showing a Contact

Once the user selects a contact the program will prompt the user to input an index which will dictate which contact is printed.

All Contacts:	supple.js:45
Linus Torvalds	supple.js:47
Ada Lovelace	supple.js:47
Alan Turing	supple.js:47

Figure 19. Showing All Contacts

In selecting to show all contacts the program will get all the names in the array and print them according to their index.

127.0.0.1:5500 says

Enter Name:

OK

Cancel

127.0.0.1:5500 says

Enter Role:

OK

Cancel

127.0.0.1:5500 says

Enter a Skill:

OK

Cancel

Figure 20. Adding a New Contact

Here in figure 20 we see a similar structure to add new contact wherein the program prompts the user for each data field needed for the array.

Contact added successfully.

[supple.js:69](#)

Figure 21. Successful Adding of Data

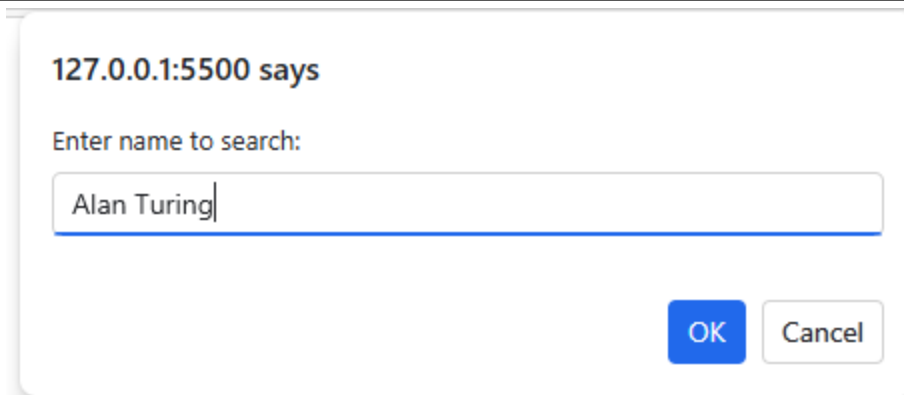


Figure 22. Searching by Name

The program is able to take the name of a person then find their records onto the array through their name.

Role: Cryptographer	supple.js:82
Status: Available	supple.js:83

Figure 23. Output of Searching by Name

Shown in figure 23 is the successful process of searching a person in the array.

Contact not found.	supple.js:94
--------------------	------------------------------

Figure 24. Output of Unsuccessful Searching by Names

Shown in figure 24 is the unsuccessful process of searching a person in the array meaning the name is not present on the array.

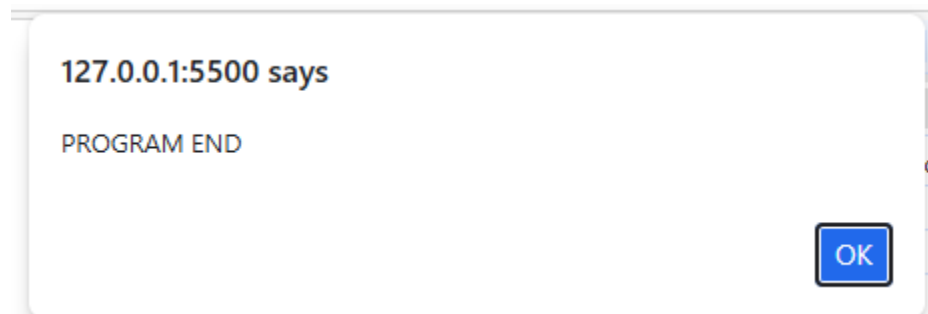


Figure 25. Quitting the Program

In figure 25 we see the alert prompt when the user types quit on the main menu.

CONCLUSION

After performing this activity I was able to learn about the structure of implementing functions in javascript. This activity taught me how to effectively create and call a function to do tasks in a program. For this activity I was able to utilize functions in a simple program for handling contact management. The use of functions allows me to organize my code and create different logical tasks in separate organisable sections. The supplementary activity helped me explore more about incorporating the different logical tasks and utilizing a functional structure for my code. I was also able to utilize loops to enable the program to endlessly run while having proper error catching.